

Static Code Analysis Lab

Course: Cybersecurity Lab

Student ID: IN2300003

Tools:

- **SAST:** SonarCloud (Online)
- **SCA:** *OWASP* Dependency Check (Docker/CLI)

Repositories:

- **Vulnerable:** “WebGoat” (Java/Spring)
- **Secure:** “*dcc-utils*” (Java) – Ministero della Salute

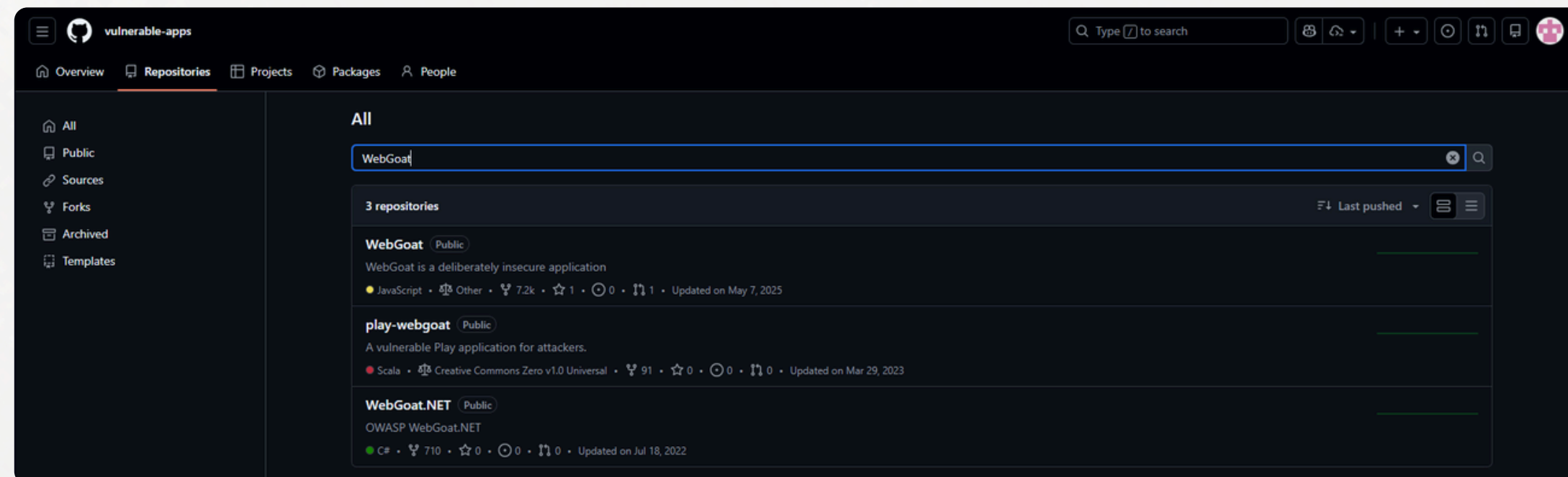
WebGoat Repo

Description:

- A deliberately insecure web application – maintained by *OWASP*
- Designed to teach web app security flaws (SQL Injection, XSS etc.)
- Found on <https://github.com/orgs/vulnerable-apps/repositories>

Why Selected:

- **Known issues:** It serves as a perfect benchmark to verify if SAST/SCA tools work properly
- **Expectation:** High detection rates and critical severity alerts



— dcc-utils Repo

Description:

- Official utility library for the “Digital COVID Certificate” in Italy
- Maintained by the Italian Ministry of Health (Ministero della Salute)
- <https://github.com/ministero-salute/dcc-utils>

Why Selected:

- **Real-World Production Code:** Represents critical public infrastructure
- **Expectation:** High code quality and security standards

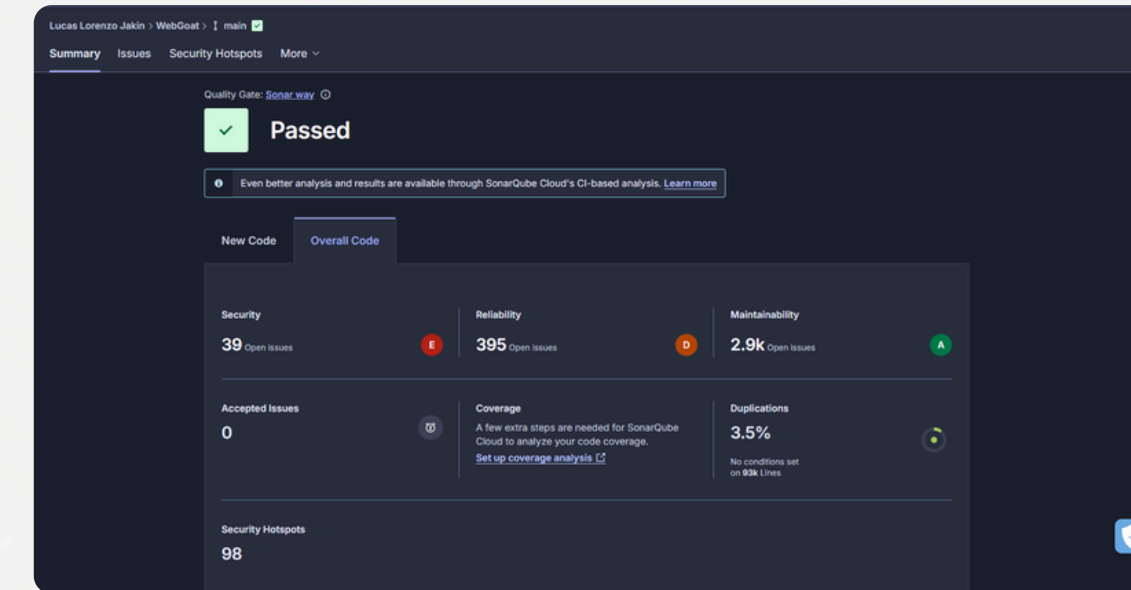
WebGoat - SAST

SonarCloud Findings:

- **Total Vulnerabilities:** 39 confirmed security issues
- **Broader Code Quality:**
 - **395 Reliability Bugs:** Potential stability risks
 - **~3,000 Code Smells:** 40 days remediation effort

Insights:

- Project “**Passed**” the Quality Gate despite the issues
- **Lesson:** Do not blindly trust a “Passed” status → Often applied only to **new code**, leaving vulnerabilities hidden unless manually inspected.



Select issues for bulk actions

☐ [Change](#) [Select issues](#) [▼](#) [▲](#) [Navigate to issue](#) [◀](#) [▶](#)

[↻](#) **35 issues** **2d 5h effort**

src/.../lessons/challenges/challenge5/Assignment5.java

☐ **Change this code to not construct SQL queries directly from user-controlled data.** [Intentionality](#)

[Security](#) [🚫 Blocker](#) [android](#) [cwe](#) [...](#) [+](#)

[🔒](#) [Open](#) [▼](#) [Not assigned](#) [▼](#) [L65](#) [•](#) [30min effort](#) [•](#) [3 years ago](#) [•](#) [🔒 Vulnerability](#) [•](#) [🚫 Blocker](#)

src/.../lessons/deserialization/InsecureDeserializationTask.java

☐ **Change this code to not deserialize user-controlled data.** [Intentionality](#)

[Security](#) [🚫 Blocker](#) [cwe](#) [+](#)

[🔒](#) [Open](#) [▼](#) [Not assigned](#) [▼](#) [L60](#) [•](#) [30min effort](#) [•](#) [3 years ago](#) [•](#) [🔒 Vulnerability](#) [•](#) [🚫 Blocker](#)

src/.../org/owasp/webgoat/lessons/jwt/JWTRefreshEndpoint.java

☐ **Revoke and change this password, as it is compromised.** [Responsibility](#)

[Security](#) [🚫 Blocker](#) [cwe](#) [+](#)

[🔒](#) [Open](#) [▼](#) [Not assigned](#) [▼](#) [L89](#) [•](#) [1h effort](#) [•](#) [3 years ago](#) [•](#) [🔒 Vulnerability](#) [•](#) [🚫 Blocker](#)

WebGoat - SCA

Vulnerable Dependencies of Risks:

- **High Volume of Risks:** A total of 286 vulnerabilities → **46 Critical** and **141 High** severity issues
- **Key Vulnerable Libraries:**
 - **H2 Database** (*CVE-2022-45868*): critical Remote Code Execution vulnerability → CVSS score of **9.8/10**
 - **Thymeleaf** (*CVE-2021-43466*): sandbox bypass issue allowing attackers to execute arbitrary code
- **Conclusion:**
 - Application is severely compromised not just by its logic, but also by its outdated libraries

Dependency-Check Report

File D:/MyLabRep/WebGoat/dependency-check-report.html

Sign in

Chat

[How to read the report](#) | [Suppressing false positives](#) | **Getting Help:** [github issues](#)

Project: WebGoat

Scan Information ([show all](#)):

- *dependency-check version:* 12.2.0
- *Report Generated On:* Thu, 15 Jan 2026 11:12:57 GMT
- *Dependencies Scanned:* 93 (90 unique)
- *Vulnerable Dependencies:* 9
- *Vulnerabilities Found:* 45
- *Vulnerabilities Suppressed:* 0
- ...

Summary

Summary of Vulnerable Dependencies ([click to show all](#))

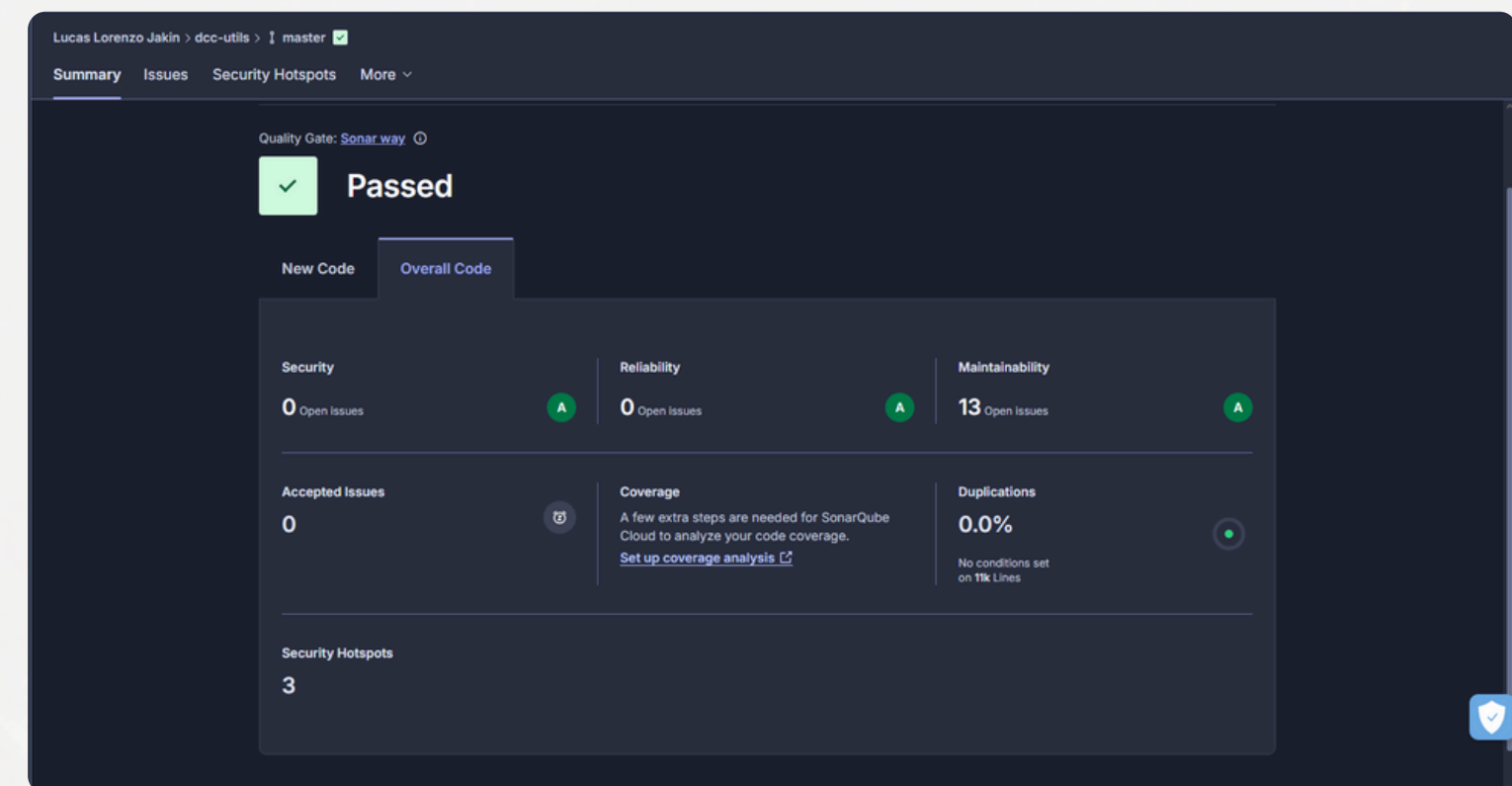
Dependency	Vulnerability IDs	Package	Highest Severity	CVE Count	Confidence	Evidence Count
bootstrap.min.js		pkg:javascript/bootstrap@3.1.1	MEDIUM	9		3
bootstrap.min.js		pkg:javascript/bootstrap@3.1.1	MEDIUM	9		3
jquery-1.10.2.min.js		pkg:javascript/jquery@1.10.2.min	MEDIUM*	6		3
jquery-2.1.4.min.js		pkg:javascript/jquery@2.1.4.min	MEDIUM*	6		3
jquery-ui-1.10.4.custom.min.js		pkg:javascript/jquery-ui@1.10.4	MEDIUM	4		3
jquery-ui-1.10.4.js		pkg:javascript/jquery-ui@1.10.3	MEDIUM	4		3
jquery-ui.min.js		pkg:javascript/jquery-ui@1.12.1	MEDIUM	4		3
jquery.min.js		pkg:javascript/jquery@3.4.1	MEDIUM*	2		3
underscore-min.js		pkg:javascript/underscore.js@1.10.2	HIGH	1		3

* indicates the dependency has a known exploited vulnerability

— dcc_utils - SAST

SonarCloud Findings:

- **Security Issues:** 0 Detected → Code is secure against standard static analysis patterns
- **Maintainability:** 13 Issues (Code Smells)
- **Analysis:**
 - Code is safe from vulnerabilities like *SQLI* or *XSS*
 - However “Clean” doesn’t always mean “Perfect”
 - Still Maintainability issues



— dcc_utils - SCA

Dependency Status:

- **Result:** Report shows minimal or zero critical vulnerabilities
- **Analysis:**
 - As a government project, dependencies are kept **up-to-date** because of **security regulations**
- **Comparison:**
 - Clear contrast to WebGoat → Demonstrates good supply chain security practices

Conclusions

SAST Tool:

- Essential for finding bugs in *our code*
- **False Negative:** Cannot understand “business” logic
 - If app allows the user to buy items at 0€, then SolarCloud might not catch this “error” → it is a **syntax error**

SCA Tool:

- Essential for finding vulnerabilities in the external libraries
- **False Positive:** Often flags libraries as “Vulnerable” even if specific vulnerable function is never used by the application
 - E.g. **LogBack Core** vulnerability → **Human verification required**

Final Thought:

- Using both tools is necessary → SAST secures the **logic** & SCA secures the **supply chain**